

Manuale Completo di Assembly x86-64 (Sintassi Intel)

NECKER

3 Dicembre 2025

Indice

1	I Registri in x86-64	2
1.1	Registri Generali di Scopo (GPR)	2
1.2	Esempio Schematico di Sovrascrittura dei Registri	2
1.2.1	Risultato dell'Esecuzione (Output Visivo)	3
1.3	Regola di Azzeramento Implicito (Implicit Zero-Extension)	3
1.4	Registri Speciali	4
2	Unità di Misura della Memoria	4
3	Istruzioni Fondamentali	4
3.1	Movimento e Stack	4
3.2	Aritmetica su Indirizzi (LEA)	4
3.3	Aritmetica Base	5
3.4	Moltiplicazione e Divisione	5
3.5	Operazioni Logiche e Bitwise	5
3.6	Controllo del Flusso	5
3.7	Istruzioni di Shift e Rotazione (Bitwise)	7
4	Accesso ai Registri Tramite Indirizzi	7
5	Syscall	9
5.1	Informazioni extra utili	12
5.2	Definizione di Opcode	12
5.3	Insidia 1: Lunghezza Variabile delle Istruzioni	13
5.4	Insidia 2: Corretto Indirizzamento e Dimensionamento	13
6	Dimensioni istruzioni	13

1 I Registri in x86-64

I registri sono le unità di storage veloci all'interno della CPU. In x86-64, ci sono diversi tipi: registri generali (GPR), registri di segmento, registri di controllo, e registri speciali. Qui ci concentriamo sui più importanti per la programmazione Assembly di base.

1.1 Registri Generali di Scopo (GPR)

Ci sono 16 registri generali a 64 bit. Ognuno ha sottoregistri per accedere a porzioni più piccole dei bit. Scrivere in un sottoregistro può influenzare il registro completo, seguendo regole specifiche.

Registro 64-bit	32-bit	16-bit	8-bit Basso	8-bit Alto	Descrizione Tipica
RAX	EAX	AX	AL	AH	Accumulatore: Usato per aritmetica e ritorno funzioni.
RBX	EBX	BX	BL	BH	Base: Per indici e puntatori generali.
RCX	ECX	CX	CL	CH	Contatore: Usato in loop e shift.
RDX	EDX	DX	DL	DH	Dati: Estensione per moltiplicazioni/divisioni.
RSI	ESI	SI	SIL	-	Sorgente Indice: Per stringhe e array (sorgente).
RDI	EDI	DI	DIL	-	Destinazione Indice: Per stringhe e array (destinazione).
RBP	EBP	BP	BPL	-	Base Pointer: Per frame stack in funzioni.
RSP	ESP	SP	SPL	-	Stack Pointer: Punta alla cima dello stack.
R8	R8D	R8W	R8B	-	Registro generale esteso (da R8 a R15).
R9	R9D	R9W	R9B	-	Registro generale esteso.
R10	R10D	R10W	R10B	-	Registro generale esteso.
R11	R11D	R11W	R11B	-	Registro generale esteso.
R12	R12D	R12W	R12B	-	Registro generale esteso.
R13	R13D	R13W	R13B	-	Registro generale esteso.
R14	R14D	R14W	R14B	-	Registro generale esteso.
R15	R15D	R15W	R15B	-	Registro generale esteso.

Nota: Scrivere in un registro grande (RDI) sovrascrive sempre le porzioni più piccole (EDI, DIL).

Tabella 1: Gerarchia completa dei Registri Generali (GPR) in x86-64.

Note sui sottoregistri:

- I registri da RAX a RDX hanno sottoregistri a 8-bit alto (AH, BH, ecc.), ma non quelli da RSI in poi.
- I registri estesi (R8-R15) sono disponibili solo in modalità 64-bit.
- Registri e sottoregistri fanno riferimento alla stessa cella di memoria, quindi scrivere in un registro a 64-bit significa sovrascrivere tutti i sottoregistri e viceversa, scrivere in un sottoregistro significa sovrascrivere il registro superiore fino al bit max sovrascritto.

1.2 Esempio Schematico di Sovrascrittura dei Registri

I registri e i sottoregistri condividono la stessa area di memoria fisica nella CPU. Scrivere in un registro a 64 bit (come RAX) sovrascrive completamente tutti i sottoregistri associati. Al contrario, scrivere in un sottoregistro (come AL o EAX) modifica solo la porzione specifica di bit, lasciando intatti gli altri (con l'eccezione della regola di azzeramento implicito per scritture a 32 bit, che azzerà i bit 32-63).

Per illustrare visivamente, useremo una simulazione in Python che emula il comportamento del registro RAX. Il codice può essere eseguito in un ambiente Python per verificare i risultati. Di seguito, il codice simulatore e l'output generato.

1.2.1 Risultato dell'Esecuzione (Output Visivo)

Di seguito, l'output generato eseguendo il codice sopra. Mostra come ogni operazione modifichi i bit specifici del registro.

Stato iniziale:

RAX: 0x0000000000000000

EAX: 0x00000000

AX: 0x0000

AH: 0x00

AL: 0x00

Dopo `mov rax, 0x1122334455667788`:

RAX: 0x1122334455667788

EAX: 0x55667788

AX: 0x7788

AH: 0x77

AL: 0x88

Dopo `mov al, 0x99`:

RAX: 0x1122334455667799

EAX: 0x55667799

AX: 0x7799

AH: 0x77

AL: 0x99

Dopo `mov eax, 0xAABBCCDD`:

RAX: 0x00000000AABBCCDD

EAX: 0xAABBCCDD

AX: 0xCCDD

AH: 0xCC

AL: 0xDD

Dopo `mov ah, 0xEE`:

RAX: 0x00000000AABBEEDD

EAX: 0xAABBEEDD

AX: 0xEEDD

AH: 0xEE

AL: 0xDD

Dopo `mov ax, 0xFF00`:

RAX: 0x00000000AABBFF00

EAX: 0xAABBFF00

AX: 0xFF00

AH: 0xFF

AL: 0x00

1.3 Regola di Azzeramento Implicito (Implicit Zero-Extension)

Quando si scrive in un sottoregistro, il comportamento sui bit superiori varia:

- **Scrittura a 32-bit (es. `EAX: mov EAX, EBX`):** Azzerata automaticamente i bit 32-63 del registro a 64-bit.
- **Scrittura a 16-bit o 8-bit (es. `AX, AL`):** Non modifica i bit superiori al 16/8.

Questa regola è cruciale per ottimizzazioni, come calcolare moduli senza aritmetica.

1.4 Registri Speciali

1. **RIP (Instruction Pointer)**: Punta all'istruzione corrente. Modificato da salti e chiamate.
2. **RFLAGS (Flags Register)**: Contiene flag come ZF (Zero Flag), SF (Sign Flag), CF (Carry Flag), OF (Overflow Flag). Usati per segnalare comportamenti particolari come l'azzeramento del risultato dopo una somma ecc.
3. **Registri di Segmento**: CS, DS, ES, FS, GS, SS. In modalità flat (tipica), non necessari per l'accesso memoria.

2 Unità di Misura della Memoria

In Assembly, le dimensioni dei dati sono definite come segue:

Termine	Dimensione in Byte	Dimensione in Bit
Byte	1	8
Word	2	16
Double Word (Dword)	4	32
Quad Word (Qword)	8	64
Double Quad Word (DQword)	16	128

Tabella 2: Unità di misura della memoria in x86-64.

3 Istruzioni Fondamentali

Le istruzioni sono raggruppate per categoria. La sintassi è **ISTRUZIONE Destinazione, Sorgente**, dove Destinazione è spesso un registro o memoria, Sorgente un registro, memoria o immediato.

3.1 Movimento e Stack

Queste istruzioni gestiscono il trasferimento dati e lo stack (cresce verso basso).

Istruzione	Sintassi	Spiegazione
MOV	MOV Dest, Sorg	Sposta valore da Sorg (registro, memoria, immediato) a Dest. Non altera flag. Es: MOV RAX, 42.
PUSH	PUSH Sorg	Decrementa RSP di 8, salva Sorg sullo stack. Usato per salvare registri.
POP	POP Dest	Copia dallo stack a Dest, incrementa RSP di 8.

3.2 Aritmetica su Indirizzi (LEA)

L'istruzione LEA calcola un indirizzo efficace, ma **non accede alla memoria**. È spesso usata per eseguire aritmetica veloce tramite la sintassi di indirizzamento.

Istruzione	Sintassi	Spiegazione
LEA	LEA Dest, [Base + Indice*Scala + Offset]	Calcola l'indirizzo dato dall'espressione senza leggerne il contenuto. Usato per operazioni aritmetiche (moltiplicazioni per 2/4/8, somme), o per costruire indirizzi. Non modifica i flag.

3.3 Aritmetica Base

Queste modificano i flag (CF, OF, SF, ZF, ecc.).

Istruzione	Sintassi	Spiegazione
ADD	ADD Dest, Sorg	Dest = Dest + Sorg. Imposta flag per overflow/carry.
SUB	SUB Dest, Sorg	Dest = Dest - Sorg. Imposta flag.
INC	INC Dest	Dest = Dest + 1. Non altera CF.
DEC	DEC Dest	Dest = Dest - 1. Non altera CF.

3.4 Moltiplicazione e Divisione

Per numeri signed, usa IMUL/IDIV.

Istruzione	Sintassi	Spiegazione
IMUL	IMUL Dest, Sorg1, Imm	Dest = Sorg1 * Imm (3 operandi).
IMUL	IMUL Dest, Sorg	Dest = Dest * Sorg (2 operandi).
IDIV	IDIV Divisore	(RAX) / Divisore. Quoziente in RAX, resto in RDX.

3.5 Operazioni Logiche e Bitwise

Modificano bit individuali.

Istruzione	Sintassi	Spiegazione
AND	AND Dest, Sorg	Dest = Dest bitwise AND Sorg. Imposta ZF se zero.
OR	OR Dest, Sorg	Dest = Dest bitwise OR Sorg.
XOR	XOR Dest, Sorg	Dest = Dest bitwise XOR Sorg. Usato comunemente per azzerare un registro (XOR RAX, RAX → RAX=0).

Tabella 8: Tabelle di Verità per le Operazioni Logiche Fondamentali

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

3.6 Controllo del Flusso

Gestiscono salti e funzioni.

Istruzione	Sintassi	Spiegazione
CMP	CMP registro1 registro2	imposta la Zero flag a 1 o a 0 per dire ai salti condizionati se saltare o meno.
Salto Incondizionato		
JMP	JMP Etichetta / offset	Salto incondizionato . L'esecuzione continua dal nuovo indirizzo. Può essere Assoluto (indirizzo diretto) o Relativo (offset rispetto a RIP).
Salto Condizionato (Basato su Flag)		

Istruzione	Sintassi	Spiegazione
JE / JZ	JE Etichetta	Salta se Uguale / Zero Flag è settato (ZF=1).
JNE / JNZ	JNE Etichetta	Salta se Non Uguale / Non Zero Flag è azzerato (ZF=0).
JG / JNLE	JG Etichetta	Salta se Maggiore (ZF=0 e SF=0F). Per numeri con segno.
JA	JA Etichetta	Salta se Maggiore (ZF=0 e SF=0F). Per numeri senza segno.
JL / JNGE	JL Etichetta	Salta se Minore (SF $\neq$ 0F). Per numeri con segno.
JB	JB Etichetta	Salta se Minore (SF $\neq$ 0F). Per numeri senza segno.
Chiamate e Ritorno da Funzioni		
CALL	CALL Etichetta	Chiama una funzione. Salva l'indirizzo di ritorno (RIP corrente) sullo Stack (PUSH RIP), poi esegue JMP a Etichetta.
RET	RET	Ritorna da una funzione. Recupera l'indirizzo di ritorno dallo Stack (POP RIP) e salta a quell'indirizzo.
Interfacciamento con il Sistema Operativo		
SYSCALL	SYSCALL	Chiamata di Sistema. Interrompe l'esecuzione e passa il controllo al kernel. Il servizio richiesto è specificato in RAX.

1. **Salto Incondizionato (JMP):** Utilizzato per creare un loop infinito.

```
.loop_start:
; Codice del loop...
jmp .loop_start ; Salta all'etichetta .loop_start
```

2. **Salto Condizionato (JE) dopo Comparazione:** Esegue una ramificazione (IF/ELSE).

```
mov rax, 5
cmp rax, rdi ; Confronta RAX con RDI. Imposta i flag.
je .RDI_Uguale_5 ; Salta qui se ZF=1 (se RAX == RDI)

; Codice per la condizione RDI != 5
jmp .fine_controllo

.RDI_Uguale_5:
; Codice per la condizione RDI == 5

.fine_controllo:
; Continua l'esecuzione
```

3. **Chiamata e Ritorno da Funzione:** Stabilisce il flusso di esecuzione per la subroutine.

```
; ... Codice principale ...
mov rdi, 10 ; Argomento 1 (RDI)
mov rsi, 20 ; Argomento 2 (RSI)
call funzione_somma
; RAX ora contiene la somma (30)
; ... Continua ...

funzione_somma:
```

```

; RDI = 10, RSI = 20
mov rax, rdi      ; RAX = 10
add rax, rsi      ; RAX = 10 + 20 = 30
ret              ; Ritorna al punto dopo CALL

```

4. **Chiamata di Sistema (SYSCALL)**: Esegue operazioni del kernel (es. Uscita dal programma).

```

; Uscita dal programma (Syscall n. 60 in Linux x86-64)
mov rax, 60      ; Numero di Syscall (sys_exit)
mov rdi, 0       ; Codice di uscita (0 = successo)
syscall          ; Esegue la chiamata al Kernel

```

3.7 Istruzioni di Shift e Rotazione (Bitwise)

Queste istruzioni spostano i bit di un registro. Sono usate per divisioni/moltiplicazioni veloci per potenze di due e per l'isolamento dei bit.

Istruzione	Sintassi	Spiegazione
SHL	SHL Dest, Conteggio	Shift Left Logico (Moltiplicazione per 2^N). Sposta N bit a sinistra; inserisce 0 a destra.
SHR	SHR Dest, Conteggio	Shift Right Logico (Divisione Unsigned per 2^N). Sposta N bit a destra; inserisce 0 a sinistra.

Strategia di Isolamento Byte (in un registro 64-bit)

Per isolare un byte B_n in un registro 64-bit (RAX, RDI, ecc.), si usa una combinazione di SHL e SHR. La strategia è portare prima il byte B_n al bit 63, e poi riportarlo al bit 0 per azzerare i bit superiori.

Tabella 11: Posizioni dei Byte in un Registro 64-bit (QWORD)

Byte	Nome	Posizione Bit	Shift per B_0	Valore
B_0	AL/R8B	7 – 0	0	Meno Significativo (Low Byte)
B_1	AH/R9B	15 – 8	8	
B_2	R10B	23 – 16	16	
B_3	R11B	31 – 24	24	
B_4	R12B	39 – 32	32	
B_5	R13B	47 – 40	40	
B_6	R14B	55 – 48	48	
B_7	R15B	63 – 56	56	Più Significativo (High Byte)

Esempio di Isolamento (Per isolare il 5° byte, B_4):

1. `mov rax, rdi` (Copia il valore)
2. `shl rax, 24` (Sposta B_4 al bit 55)
3. `shr rax, 56` (Sposta B_4 al bit 0 e azzerare il resto)

Il valore originale di B_4 è ora il solo contenuto in AL (e RAX).

4 Accesso ai Registri Tramite Indirizzi

Normalmente, i registri della CPU vengono manipolati direttamente tramite il loro nome simbolico (RAX, RDI, ecc.). Tuttavia, i registri possono anche essere mappati a indirizzi all'interno dello spazio di memoria (Memory-Mapped Registers).

Istruzioni per l'Accesso Indiretto

Le istruzioni di accesso alla memoria vengono utilizzate per leggere o scrivere il valore di un "registro mappato":

Istruzione	Sintassi	Spiegazione
MOV	MOV Dest, [Indirizzo_Reg]	Lettura: Sposta il valore a 64 bit (QWORD) memorizzato nell'indirizzo di memoria che rappresenta il registro in un registro di destinazione (Dest).
MOV	MOV [Indirizzo_Reg], Sorg	Scrittura: Sposta il valore dal registro sorgente (Sorg) all'indirizzo di memoria che rappresenta il registro.
ADD	ADD [Indirizzo_Reg], Imm	Modifica: Incrementa direttamente il valore del registro mappato in memoria con un valore immediato (Imm).

Esempio Ipotetico

Ipotizzando che il registro RDI sia mappato all'indirizzo 0x405000:

- `mov rax, [0x405000]` → Legge il valore "simulato" di RDI in RAX.
- `mov [0x405000], rbx` → Scrive il valore di RBX nell'indirizzo mappato, modificando il "simulato" RDI.

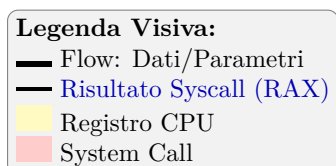
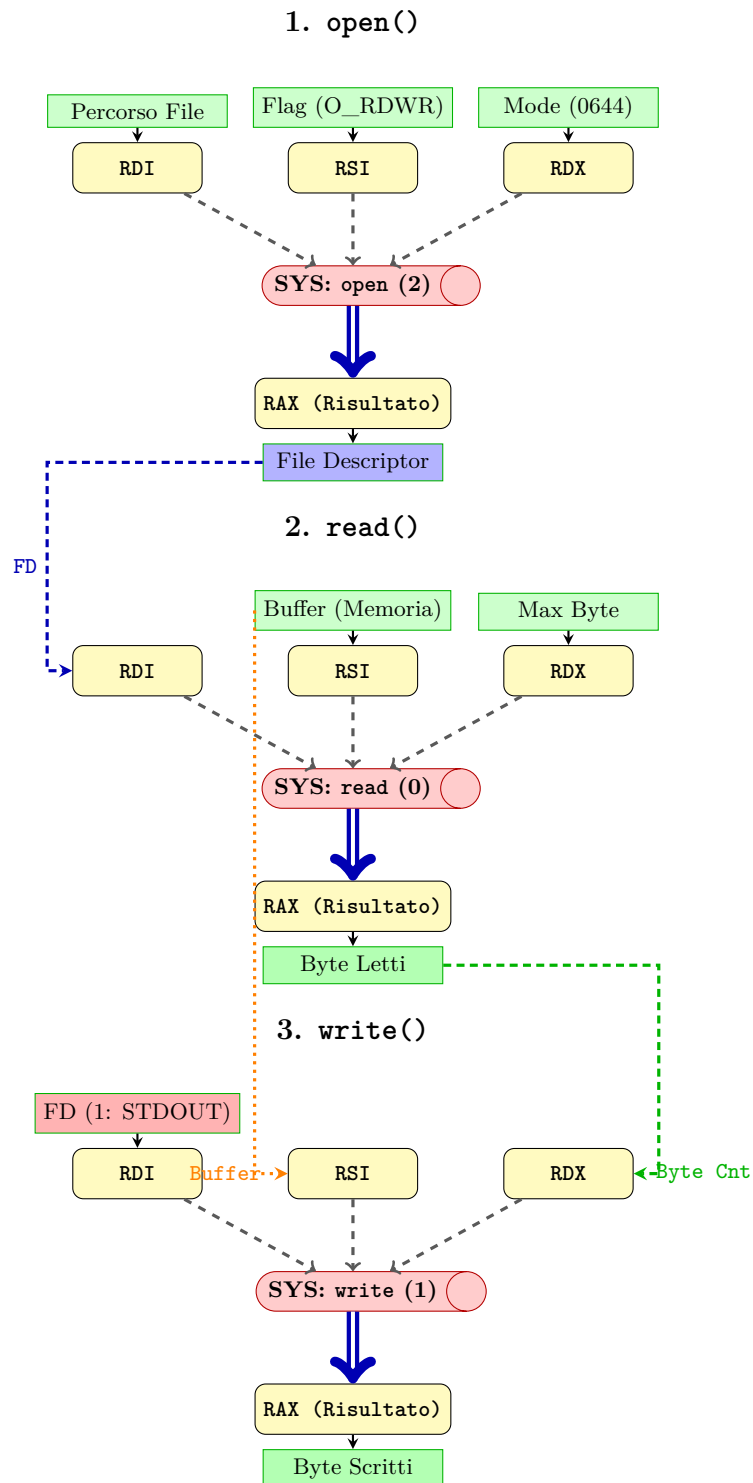
5 Syscall

Per lavorare con syscall in ambiente Linux su architettura x86-64, è essenziale conoscere il meccanismo di chiamata al kernel. Le syscall si invocano impostando il numero in `rax`, i parametri nei registri specifici (`rdi`, `rsi`, `rdx`, ecc.) e eseguendo l'istruzione `syscall`. Al ritorno, `rax` contiene il risultato (es. valore di ritorno o codice errore se negativo).

Di seguito, una tabella riassuntiva per le syscall comuni legate alla gestione file (open, read, write). La tabella indica il numero syscall, i registri principali e i valori tipici da impostare per scenari base (es. lettura di un file di testo).

Syscall	Numero (RAX)	Parametri (Registri)	Valore di Ritorno (RAX)
open	2	• RDI: Indirizzo (pathname) della stringa null-terminated del file. • RSI: Flag di accesso (es. 0 per <code>O_RDONLY</code>, 1 per <code>O_WRONLY</code>). • RDX: Permessi (mode) per la creazione (usato solo con <code>O_CREAT</code>).	Successo: File Descriptor (FD) positivo. Errore: Valore negativo (opposto del codice errore).
read	0	• RDI: File Descriptor (FD) da cui leggere. • RSI: Indirizzo del buffer dove salvare i dati letti. • RDX: Numero massimo di byte da leggere (count).	Successo: Numero di byte letti. EOF: 0. Errore: Valore negativo.
write	1	• RDI: File Descriptor (FD) su cui scrivere (es. 1 per <code>stdout</code>). • RSI: Indirizzo del buffer contenente i dati da scrivere. • RDX: Numero di byte da scrivere (count).	Successo: Numero di byte scritti. Errore: Valore negativo.
sys_chmod	90	• RDI: Indirizzo (pathname) della stringa null-terminated del file. • RSI: Mode, numero in oottale (es. 0777 dove 0 iniziale indica base ottale e modifica i permessi in r,w,e per tutti).	Successo: 0

Syscall	Numero (RAX)	Parametri (Registri)	Valore di Ritorno (RAX)
			Errore: Valore negativo (opposto del codice errore).



5.1 Informazioni extra utili

Ecco alcune note generali e best practice per usare queste syscall in shellcode o assembly low-level:

- **File descriptor (fd):** Sono interi che rappresentano file aperti. Valori standard:
 - 0: stdin (input standard, es. da tastiera).
 - 1: stdout (output standard, es. per stampare su console).
 - 2: stderr (errori standard).
 - $fd \geq 3$: Assegnati dinamicamente da open (es. per file utente come /flag).

Per stampare, usa `fd=1` in write per inviare direttamente all'output senza buffer extra.

- **Flags per open:** Definisci il modo di apertura. Esempi comuni:
 - `O_RDONLY` (0): Solo lettura, ideale per file come flag.
 - `O_WRONLY` (1): Solo scrittura.
 - `O_RDWR` (2): Lettura e scrittura.

Imposta con `xor rsi, rsi` per 0 (null-free).

- **Gestione errori:** Se `rax < 0` dopo syscall, è un errore (es. `-ENOENT` per file non trovato). In shellcode semplici, ignoralo se assumi successo, ma in codice robusto controlla con `test rax, rax; js errore`.
- **Buffer e stack:** Per read/write, alloca spazio su stack con `sub rsp, N` (es. `0x100` byte). Usa `mov rsi, rsp` per puntatore. Evita overflow: `rdx` in read/write deve matchare la size allocata.
- **Null-free e position-independent:** In constraint come no byte `0x00`, costruisci valori con `xor/inc/shl` invece di `mov` diretti. Per stringhe, usa little-endian (es. hex invertito per dword).

Opcode, Codifica delle Istruzioni e Insidie Comuni

5.2 Definizione di Opcode

L'**Opcode** (Operation Code) è il primo campo di una qualsiasi istruzione in codice macchina e identifica in modo univoco l'operazione che la CPU deve eseguire (es. `MOV`, `ADD`, `JMP`).

- L'Opcode è solitamente un codice esadecimale di 1 o 2 byte (es. `B8` per `MOV RAX, Imm64`).
- È ciò che l'assembler traduce dal testo (`mov`) al codice binario che la CPU comprende.

Mappatura Opcode per Istruzioni Fondamentali

Questa tabella mostra gli Opcode (codici esadecimali) dei tipi di istruzione più frequentemente usati, specialmente quelli relativi al trasferimento e al controllo del flusso.

Istruzione	Sintassi (Esempio)	Opcode Iniziale (H)	Lunghezza Tipica (Byte)
Trasferimento Dati (MOV)			
MOV Reg → Reg	<code>mov rax, rbx</code>	48 89	3
MOV Imm → Reg 64-bit	<code>mov rax, 0x...</code>	48 B8	10
MOV Mem → Reg	<code>mov rax, [rdi]</code>	48 8B 07	3
MOV Mem Stack → Reg	<code>mov rdi, [rsp]</code>	48 8B 3C 24	4
Aritmetica e Logica			
ADD Reg → Reg	<code>add rax, rbx</code>	48 01	3
SUB Mem → Reg	<code>sub [rsp], rdi</code>	48 29	4
Controllo del Flusso			
JMP Relativo 32-bit	<code>jmp Etichetta</code>	E9	5 (1 Opcode + 4 Offset)
JMP Assoluto (via Reg)	<code>jmp rax</code>	FF E0	2

Istruzione	Sintassi (Esempio)	Opcode Iniziale (H)	Lunghezza Tipica (Byte)
CALL Relativo 32-bit	<code>call Etichetta</code>	E8	5 (1 Opcode + 4 Offset)
Jcc (Condizionale)	<code>je Etichetta</code>	74	2 (1 Opcode + 1 Offset 8-bit)
Altre Istruzioni			
NOP (No Operation)	<code>nop</code>	90	1

Nota sulla Codifica (Prefissi 48): Il prefisso **48** (chiamato **REX.W**) appare in molte istruzioni a 64 bit. Questo prefisso informa la CPU che l'operazione deve essere eseguita su un registro a 64 bit (Quad Word) anziché sulla dimensione predefinita a 32 bit, modificando così l'Opcode base.

quindi se usiamo registri a 32 bit l'opcode non sarà più 40 es: `mov eax, edx`.

5.3 Insidia 1: Lunghezza Variabile delle Istruzioni

Nell'architettura x86-64, le istruzioni non hanno una lunghezza fissa. La lunghezza varia in base all'Opcode, al numero di operandi e alla dimensione degli operandi (8, 16, 32 o 64 bit).

- **Esempio:** NOP occupa 1 byte (90). Un JMP relativo (salto a 32 bit) occupa 5 byte (E9 + 4 byte di offset).
- **Come Evitare Errori:** Nei salti relativi (JMP, CALL), è fondamentale conoscere l'esatta lunghezza dell'istruzione per calcolare correttamente l'offset di destinazione. È sempre preferibile usare le etichette (`.label`) e lasciare il calcolo dell'offset all'assembler.

5.4 Insidia 2: Corretto Indirizzamento e Dimensionamento

La CPU necessita di sapere esattamente quanti byte leggere o scrivere dalla memoria.

- **Insidia POP/PUSH:** Le istruzioni PUSH e POP modificano sempre il registro Stack Pointer (RSP). Per leggere o modificare il valore in cima allo stack **senza** spostare RSP, è necessario accedere alla memoria in modo diretto:

```
mov rax, qword ptr [rsp] (Leggi il contenuto senza cambiare RSP)
```

- **Insidia QWORD PTR:** Se la destinazione è un registro (es. `mov rax, [addr]`), la dimensione dei dati è implicita (64 bit, QWORD). Se la destinazione è un indirizzo di memoria o la dimensione è ambigua, è obbligatorio specificare la dimensione con direttive come QWORD PTR:

```
sub qword ptr [rsp], rdi (Operazione su 8 byte di memoria)
```

6 Dimensioni istruzioni

Precisazione sulla Lunghezza di MOV (RAX vs EAX)

La differenza nella lunghezza dell'istruzione MOV tra i registri a 64 bit (RAX) e quelli a 32 bit (EAX) è interamente dovuta alla dimensione del **Dato Immediato** (la costante che si sta spostando) che il processore deve caricare:

1. **mov rax, imm64 (10 byte):** Per caricare un intero completo a 64 bit (fino a $2^{64} - 1$) in RAX, il processore necessita di:
 - 1 Byte: Prefisso REX (necessario per abilitare l'operazione a 64 bit).
 - 1 Byte: Opcode (0xB8).
 - 8 Byte: Il **Dato Immediato** a 64 bit da caricare.
2. **mov eax, imm32 (5 byte):** Per caricare un intero a 32 bit (fino a $2^{32} - 1$) in EAX, il processore non usa il prefisso REX per il dato immediato ma usa lo schema di codifica nativo a 32 bit:
 - 1 Byte: Opcode (0xB8).
 - 4 Byte: Il **Dato Immediato** a 32 bit.

Poiché lo shellcode è limitato in lunghezza, i programmatori evitano sempre la costosa istruzione da 10 byte, preferendo l'istruzione da 5 byte se il valore da caricare rientra nei 32 bit, o costruendo il valore in RAX attraverso operazioni più corte (come `xor` e `add/push`).

Tabella 15: Lunghezza in Byte delle Istruzioni Comuni nello Shellcode (x64)

Istruzione Esempio	Lunghezza Tipica	Dettagli e Componenti
I. Manipolazione Dati e Stack		
push reg (e.g., push rsp)	1 Byte	Istruzione più breve. Codifica l'operazione in un singolo byte (Opcode + reg implicito poiché ogni reg ha il suo Opcode per push).
pop reg (e.g., pop rdi)	1 Byte	Molto efficiente in byte per recuperare indirizzi. (come push)
push imm32 (e.g., push 0x...)	5 Byte	0x68 (Opcode) + 4 Byte di dato immediato.
mov reg, reg (e.g., mov rdi, rax)	3 Byte	Richiede un prefisso REX (1B) per operare a 64 bit + Opcode/ModR/M (2B).
inc reg (e.g., inc rax)	3 Byte	Richiede prefisso REX (1B). Spesso sostituito da <code>add reg, 1</code> in codifiche vecchie.
nop	1 Byte	Nessuna Operazione (0x90). Utilizzato per padding.
II. Dati Immediati e Salto		
mov rax, imm64	10 Byte	Opcode 0xB8 + Prefisso REX (1B) + 8 Byte di dato immediato. (Vedi precisazione sotto).
mov eax, imm32	5 Byte	Opcode 0xB8 + 4 Byte di dato immediato. Efficiente se il valore si adatta a 32 bit.
xor reg, reg (e.g., xor rax, rax)	3 Byte	Molto usato per azzerare registri (RAX = 0) in modo efficiente in byte.
III. Controllo del Flusso		
syscall	2 Byte	Istruzione fissa (0F 05) per eseguire chiamate al kernel.
jmp rel8	2 Byte	Salto incondizionato relativo con offset a 8 bit (se il salto dista al massimo 127 byte, 1 byte di offset = 8 bit = 127 opzioni, si salta a byte). Più compatto di <code>jmp rel32</code> .
jmp rel32	5 Byte	Salto incondizionato relativo con offset a 32 bit (stesso ragionamento ma con 4 byte di offset). Più compatto di <code>jmp rel64</code> .
jmp rel64	6+ Byte	Salto incondizionato relativo con offset a 64 bit (dipende da quanto distante).
jne/je rel8	2 Byte	Salto condizionato con offset a 8 bit (es. dopo <code>cmp</code> o <code>test</code>).